

Day 4 Feature Engineering

Yesterday 12:38 PM

You are my **Machine Learning Mentor, Data Scientist Interview Trainer, and Case Study Instructor.**

I am following a **60-day / 2-month Machine Learning roadmap** to become **job-ready for a Data Scientist role** And its fourth day and teach me according to it

Cover topic such as theory mathematical intuition ,advantages ,Disadvantages , applications ,python code practice ,1project ,30 mcq questions and Potential interview questions

Show moreShow less

Day 4 — Feature Engineering & Feature Selection

You are now moving from **“cleaning data”** → **“making data useful for ML.”**

For a Data Scientist, this is one of the most important practical skills because **good features can improve a mediocre model, while poor features can make even a sophisticated model perform badly.**

Day 4 Learning Objectives

By the end of today, you should be able to:

1. Understand what features are and why they matter.
2. Differentiate **feature engineering vs feature selection.**
3. Perform numerical feature transformations.
4. Create categorical features.
5. Extract useful date/time features.
6. Create interaction and aggregation features.
7. Understand feature selection.
8. Use correlation, variance threshold and mutual information.
9. Understand RFE and L1 regularization.
10. Detect multicollinearity.
11. Understand VIF.
12. Build a feature-engineering pipeline.
13. Apply everything to a real-world case study.

1. What Is a Feature?

A **feature** is an input variable used by an ML model to make predictions.

Suppose we want to predict whether a customer will default on a loan.

Age	Income	Loan Amount	Credit Score	Default
25	40,000	200,000	650	1
45	100,000	300,000	780	0
31	50,000	500,000	610	1

Here:

Features (X):

- Age
- Income
- Loan Amount
- Credit Score

Target (y):

- Default

Mathematically:

$$X = [x_1 \quad x_2 \quad x_3 \quad \dots \quad x_p]$$

The model attempts to learn:

$$y = f(X)$$

2. Feature Engineering

Feature engineering means **creating better input variables from existing raw data**.

Example

Suppose:

Annual Income = ₹600,000
Loan Amount = ₹300,000

Instead of giving the model only these two variables, we can create:

$$DebtIncomeRatio = \frac{LoanAmount}{Income}$$

Therefore:

$$DebtIncomeRatio = \frac{300000}{600000} = 0.5$$

This new feature may be more informative than either variable individually.

Why?

The model doesn't automatically understand every meaningful relationship in your business data.

Feature engineering injects **domain knowledge** into the dataset.

3. Feature Engineering vs Feature Selection

This is a very common interview question.

Feature Engineering	Feature Selection
Creates new features	Selects existing features
Changes representation	Removes unnecessary features
Example: Income/Loan	Example: remove Age
Can increase feature count	Usually decreases feature count
Uses domain knowledge	Uses statistical/model-based methods

Example

Original:

```
Age
Income
Loan Amount
Credit Score
```

Feature engineering:

```
Debt-to-Income Ratio
Age Group
Loan-to-Income Ratio
```

Feature selection:

```
Remove Age
Remove highly redundant variable
Keep Income
Keep Credit Score
Keep DTI
```

4. Numerical Feature Engineering

4.1 Log Transformation

Suppose income looks like:

```
20,000
25,000
```

```
30,000
50,000
100,000
500,000
5,000,000
```

This is highly right-skewed.

A log transformation compresses large values:

$$X' = \log(1 + X)$$

We generally use `log1p()` when zero values may exist.

Python

```
Python
import numpy as np

df["log_income"] = np.log1p(df["income"])
```

Advantages

- Reduces skewness
- Compresses extreme values
- Can make relationships more linear

Disadvantages

- Less interpretable than original values
- Not appropriate for arbitrary negative-valued variables
- Transformation must be applied consistently to future data

Applications

- Income
- Transaction amount
- Population
- Revenue
- Claim amount

5. Polynomial Features

Suppose:

$$y = 3x^2 + 2x + 5$$

A simple linear model:

$$y = \beta_0 + \beta_1 x$$

cannot directly model this curved relationship.

We can create:

$$x^2$$

and then:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

Python

```
Python
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

For:

```
X = [2, 3]
```

degree 2 can generate:

```
x1
x2
x12
x1*x2
x22
```

Advantages

- Captures nonlinear relationships
- Useful for linear models

Disadvantages

- Feature explosion
- Increased overfitting risk
- Can create multicollinearity

6. Binning / Discretization

Instead of:

```
Age = 27
Age = 43
Age = 68
```

we can create:

```
18-30 → Young
31-50 → Middle
51+ → Senior
```

Python:

```
Python
bins = [0, 30, 50, 100]
labels = ["Young", "Middle", "Senior"]

df["age_group"] = pd.cut(
    df["age"],
    bins=bins,
    labels=labels
)
```

Advantages

- Easy interpretation
- Can capture nonlinear effects
- Useful for business reporting

Disadvantages

- Loses information
- Choice of boundaries can be arbitrary

7. Categorical Feature Engineering

Suppose:

```
City
Mumbai
Pune
Delhi
Mumbai
Pune
```

One-Hot Encoding

Convert:

```
Mumbai
Pune
Delhi
```

into:

	Mumbai	Pune	Delhi
	1	0	0

	Mumbai	Pune	Delhi
	0	1	0
	0	0	1

Python:

```
Python
pd.get_dummies(df, columns=["city"], drop_first=True)
```

Advantages

- Simple
- Interpretable
- Works well with many classical ML algorithms

Disadvantages

High-cardinality categories can create thousands of columns.

For example:

```
Customer_ID
Product_ID
ZIP_Code
```

may have enormous cardinality.

8. Frequency Encoding

Instead of one-hot encoding:

```
Mumbai → 0.40
Pune → 0.25
Delhi → 0.20
```

The value represents category frequency.

```
Python
freq = df["city"].value_counts(normalize=True)
df["city_frequency"] = df["city"].map(freq)
```

Useful for:

- High-cardinality categorical variables
- Customer data
- Product data
- Location data

9. Target Encoding

Suppose:

City	Default Rate
Mumbai	8%
Pune	5%
Delhi	12%

We can encode each city using its target mean.

But there is a **major danger: target leakage**.

If you calculate target encoding using the entire dataset before train/test splitting, information from the test set can enter your training features.

Correct principle

```
Split
↓
Train encoding
↓
Apply encoding to validation/test
```

For cross-validation, target encoding should be performed **inside each training fold**.

This is an important Data Scientist interview topic.

10. Date/Time Feature Engineering

Raw:

```
2026-08-12 14:35:20
```

can become:

```
year
month
day
day_of_week
hour
is_weekend
quarter
```

Python:

```
Python
df["date"] = pd.to_datetime(df["date"])

df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["day_of_week"] = df["date"].dt.dayofweek
df["hour"] = df["date"].dt.hour

df["is_weekend"] = (
    df["day_of_week"] >= 5
).astype(int)
```

Example applications

E-commerce:

```
hour → shopping behavior
day_of_week → weekday/weekend behavior
month → seasonal demand
```

Banking:

```
transaction hour
month
days since account opening
```

11. Interaction Features

Sometimes two variables together provide more information than separately.

Example:

$$Risk = Income \times LoanAmount$$

or:

$$DTI = \frac{LoanAmount}{Income}$$

Python:

```
Python
df["loan_income_ratio"] = (
    df["loan_amount"] / df["income"]
)
```

Another example:

```
Python
df["income_per_dependent"] = (
    df["income"] / (df["dependents"] + 1)
)
```

12. Aggregation Features

This is extremely important in real Data Science projects.

Suppose transaction-level data:

Customer	Transaction	Amount
A	T1	500
A	T2	1000
A	T3	2000
B	T4	300

We can create customer-level features:

```
total_spend
average_spend
number_of_transactions
maximum_transaction
minimum_transaction
```

Python:

```
Python
customer_features = df.groupby("customer_id").agg(
    total_spend=("amount", "sum"),
    avg_spend=("amount", "mean"),
    transaction_count=("amount", "count"),
    max_spend=("amount", "max")
).reset_index()
```

These are powerful features for:

- Fraud detection
- Credit risk
- Customer churn
- Recommendation systems
- Marketing

13. Feature Selection

Suppose you have:

1,000 features

but only 50 are useful.

Feature selection attempts to identify the informative variables.

Benefits:

- Faster training
- Lower overfitting risk
- Better interpretability
- Lower memory usage
- Easier deployment

14. Filter Methods

Filter methods select features **before training the model**.

Examples:

- Correlation
- Variance Threshold
- Mutual Information
- Chi-square

14.1 Variance Threshold

A feature that barely changes:

```
1
1
1
1
1
1
1
```

probably provides little information.

```
Python
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.01)

X_selected = selector.fit_transform(X)
```

15. Correlation-Based Selection

Suppose:

$$\text{corr}(X_1, X_2) = 0.98$$

These variables contain highly similar information.

Keeping both may be unnecessary, particularly for models sensitive to multicollinearity.

```
Python
corr_matrix = df.corr(numeric_only=True)

print(corr_matrix)
```

A common approach:

```
Python
upper = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

to_drop = [
    column for column in upper.columns
    if any(abs(upper[column]) > 0.90)
]

df_reduced = df.drop(columns=to_drop)
```

⚠ Don't blindly remove every highly correlated feature. Domain meaning and model type matter.

16. Mutual Information

Mutual Information measures how much knowing one variable tells us about another.

Conceptually:

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

If:

$$I(X; Y) = 0$$

the variables are statistically independent under the measure.

Python:

```
Python
from sklearn.feature_selection import mutual_info_classif

mi = mutual_info_classif(X, y)

mi_scores = pd.Series(mi, index=X.columns)

print(mi_scores.sort_values(ascending=False))
```

Advantage

Can detect nonlinear relationships.

Disadvantage

Doesn't directly tell you the causal importance of a variable.

17. Wrapper Methods

Wrapper methods repeatedly train a model using different feature subsets.

Examples:

- Forward Selection
- Backward Elimination
- RFE

18. Recursive Feature Elimination — RFE

RFE:

1. Train model.
2. Calculate feature importance.
3. Remove least important feature(s).
4. Train again.
5. Continue until desired number remains.

Python:

```
Python
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

rfe = RFE(
    estimator=model,
    n_features_to_select=5
)

rfe.fit(X, y)

selected_features = X.columns[rfe.support_]

print(selected_features)
```

Disadvantages

- Computationally expensive
- Depends on chosen model

19. Embedded Methods

Feature selection happens **during model training**.

Important examples:

L1 Regularization

Lasso-style penalty:

$$Loss = MSE + \lambda \sum |\beta_j|$$

The L1 penalty can push some coefficients toward exactly zero.

Therefore:

```
β1 = 2.3
β2 = 0
β3 = -1.8
β4 = 0
```

Features corresponding to zero coefficients can potentially be removed.

Python:

```
Python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    penalty="l1",
    solver="liblinear"
)

model.fit(X, y)
```

20. Tree-Based Feature Importance

Decision trees and ensemble models can provide feature importance.

```
Python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

model.fit(X, y)

importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print(importance)
```

⚠ Important interview point:

Tree feature importance is not automatically causal importance.

Also, impurity-based importance can be biased toward certain feature types.

21. Multicollinearity

Multicollinearity occurs when independent variables are strongly related to each other.

Example:

```
Annual Income
Monthly Income
Annual Salary
```

These may carry almost identical information.

For linear regression:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

high correlation among X variables can make coefficient estimates unstable.

Symptoms

- Unstable coefficients

- Large standard errors
- Difficult interpretation
- Unexpected coefficient signs

22. VIF — Variance Inflation Factor

VIF measures how much the variance of a regression coefficient is inflated due to multicollinearity.

$$VIF_j = \frac{1}{1 - R_j^2}$$

where R_j^2 comes from regressing feature X_j against the other predictors.

Interpretation is context-dependent, but commonly:

VIF	Interpretation
≈ 1	Little multicollinearity
1–5	Usually acceptable
>5	Potential concern
>10	Often considered severe

Python:

```
Python
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif["feature"] = X.columns

vif["VIF"] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]

print(vif)
```

23. Complete Feature Engineering Pipeline

A typical real-world pipeline:

```
Raw Data
↓
Data Cleaning
↓
Train/Test Split
↓
Feature Engineering
↓
Encoding
↓
Scaling
↓
Feature Selection
↓
Model Training
```

↓ Evaluation

Critical rule

Never allow test-set information to influence feature engineering decisions.

This prevents:

Data Leakage 🚨



Python Practice

Let's create a small dataset.

Python

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "age": [22, 35, 45, 52, 29, 41],
    "income": [30000, 50000, 80000, 120000, 40000, 90000],
    "loan_amount": [100000, 150000, 200000, 250000, 120000, 300000],
    "city": ["Mumbai", "Pune", "Mumbai", "Delhi", "Pune", "Delhi"],
    "dependents": [0, 2, 1, 3, 1, 2]
})

df
```

Create DTI

Python

```
df["loan_income_ratio"] = (
    df["loan_amount"] / df["income"]
)
```

Log income

Python

```
df["log_income"] = np.log1p(df["income"])
```

Age groups

Python

```
df["age_group"] = pd.cut(
    df["age"],
    bins=[0, 30, 50, 100],
    labels=["Young", "Middle", "Senior"]
)
```

Income per dependent

Python

```
df["income_per_dependent"] = (
    df["income"] / (df["dependents"] + 1)
)
```

One-hot encode

```
Python
df_encoded = pd.get_dummies(
    df,
    columns=["city"],
    drop_first=True
)
```



Day 4 Case Study — Credit Risk Feature Engineering

This is particularly useful for a Data Scientist because it resembles a real banking problem.

Business Problem

A bank wants to predict:

Will a borrower default on a loan?

Raw variables:

```
age
annual_income
loan_amount
employment_years
number_of_dependents
previous_defaults
```

Instead of directly using everything, engineer:

Feature 1

$$\text{LoanIncomeRatio} = \frac{\text{LoanAmount}}{\text{AnnualIncome}}$$

Feature 2

$$\text{IncomePerDependent} = \frac{\text{Income}}{\text{Dependents} + 1}$$

Feature 3

Age Group

Feature 4

Employment stability

Feature 5

Previous default indicator

Feature 6

Loan burden category

Example:

```
Python
df["loan_income_ratio"] = (
    df["loan_amount"] / df["annual_income"]
)

df["income_per_dependent"] = (
    df["annual_income"] /
    (df["number_of_dependents"] + 1)
)

df["has_previous_default"] = (
    df["previous_defaults"] > 0
).astype(int)

df["loan_burden"] = pd.cut(
    df["loan_income_ratio"],
    bins=[0, 0.3, 0.6, 1, np.inf],
    labels=["Low", "Medium", "High", "Very High"]
)
```

End-to-End Mini Project

Customer Churn Feature Engineering

Objective

Predict whether a customer will leave a telecom company.

Raw Data

```
customer_id
age
monthly_charges
total_charges
tenure
contract
payment_method
support_calls
last_login
churn
```

Step 1 — Create tenure groups

```
Python
df["tenure_group"] = pd.cut(
    df["tenure"],
    bins=[0, 12, 24, 48, np.inf],
    labels=["New", "1-2 Years", "2-4 Years", "4+ Years"]
)
```

Step 2 — Average monthly spending

Python

```
df["avg_monthly_spend"] = (
    df["total_charges"] /
    (df["tenure"] + 1)
)
```

Step 3 — Support intensity

Python

```
df["support_intensity"] = (
    df["support_calls"] /
    (df["tenure"] + 1)
)
```

Step 4 — Encode categorical features

Python

```
df = pd.get_dummies(
    df,
    columns=["contract", "payment_method"],
    drop_first=True
)
```

Step 5 — Separate X and y

Python

```
X = df.drop(
    columns=["customer_id", "churn"]
)

y = df["churn"]
```

Step 6 — Train/test split

Python

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Step 7 — Feature selection

Python

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import mutual_info_classif

selector = SelectKBest(
    score_func=mutual_info_classif,
    k=8
)
```

```
X_train_selected = selector.fit_transform(
    X_train,
    y_train
)

X_test_selected = selector.transform(X_test)
```

Notice the important pattern:

```
fit → training data
transform → test data
```

Never:

```
Python
selector.fit(X, y)
```

before the split.

Mathematical Intuition — The Big Picture

Imagine your raw data is:

$$X = [Age, Income, Loan]$$

You engineer:

$$X' = [Age, Income, Loan, DTI, LogIncome, AgeGroup]$$

You are transforming the representation:

$$X \rightarrow \phi(X)$$

where ϕ is your feature-engineering function.

The ML model then learns:

$$y = f(\phi(X))$$

The goal isn't necessarily to create **more features**.

The goal is to create **more informative representations**.

Advantages & Disadvantages

Technique	Advantages	Disadvantages
Log transform	Reduces skew	Less intuitive
Polynomial	Captures nonlinear patterns	Feature explosion
Binning	Easy interpretation	Information loss
One-hot	Simple	High dimensionality
Frequency encoding	Handles high cardinality	May lose semantics

Technique	Advantages	Disadvantages
Target encoding	Powerful	Leakage risk
Interaction	Captures relationships	Can increase complexity
Aggregation	Very powerful	Requires correct grouping
Correlation	Fast	Mainly linear relationships
Mutual information	Captures nonlinear dependency	Doesn't imply causality
RFE	Model-aware	Computationally expensive
L1	Automatic sparsity	Sensitive to scaling/correlation
Tree importance	Captures nonlinear effects	Can be biased

When Should You Use What?

Small numerical dataset

Try:

Correlation
Mutual Information
L1

High-dimensional dataset

Try:

Variance Threshold
L1
Tree-based selection

Nonlinear relationships

Try:

Mutual Information
Tree models
Polynomial features

High-cardinality categorical variables

Consider:

Frequency encoding
Target encoding
Hashing

with careful leakage control.

30 MCQs — Day 4

1. What is feature engineering?

- A. Removing duplicate rows
- B. Creating useful features from data
- C. Deploying a model
- D. Measuring accuracy

Answer: B

2. Which is a derived feature?

- A. Age
- B. Income
- C. Loan-to-income ratio
- D. Customer ID

Answer: C

3. Log transformation is particularly useful for:

- A. Highly skewed numerical variables
- B. Removing categorical variables
- C. Creating labels
- D. Splitting datasets

Answer: A

4. `np.log1p(x)` calculates:

- A. $\log(x-1)$
- B. $\log(x)$
- C. $\log(1+x)$
- D. $1/\log(x)$

Answer: C

5. One-hot encoding is used primarily for:

- A. Numerical scaling
- B. Categorical variables
- C. Missing values
- D. Outlier removal

Answer: B

6. What is the major problem with one-hot encoding high-cardinality variables?

- A. Underfitting
- B. Too few features
- C. Feature explosion
- D. Data normalization

Answer: C

7. Target encoding can cause:

- A. Data leakage
- B. Underflow
- C. Gradient explosion
- D. Class removal

Answer: A

8. Date feature engineering can extract:

- A. Month
- B. Day of week
- C. Hour
- D. All of the above

Answer: D

9. Which is an interaction feature?

- A. Age
- B. Income
- C. Income $\times$ Loan Amount
- D. Customer ID

Answer: C

10. Feature selection primarily aims to:

- A. Increase duplicate variables
- B. Select useful features
- C. Increase missing values
- D. Create labels

Answer: B

11. Which is a filter method?

- A. RFE
- B. Correlation

- C. Random Forest
- D. Neural network

Answer: B

12. Variance Threshold removes features with:

- A. High variance
- B. Low variance
- C. High target values
- D. Missing targets

Answer: B

13. Mutual Information measures:

- A. Dataset size
- B. Dependency/information relationship
- C. Model accuracy
- D. Training time

Answer: B

14. RFE stands for:

- A. Random Feature Evaluation
- B. Recursive Feature Elimination
- C. Regression Feature Estimation
- D. Recursive Function Evaluation

Answer: B

15. RFE is a:

- A. Wrapper method
- B. Filter method
- C. Scaling method
- D. Encoding method

Answer: A

16. L1 regularization can:

- A. Force some coefficients to zero
- B. Increase every coefficient
- C. Remove the target
- D. Create missing values

Answer: A

17. Which regularization promotes sparsity?

- A. L1
- B. L2
- C. Batch normalization
- D. Dropout

Answer: A

18. Multicollinearity occurs between:

- A. Target and predictions
- B. Independent variables
- C. Training and test labels
- D. Rows only

Answer: B

19. VIF is used to detect:

- A. Class imbalance
- B. Multicollinearity
- C. Missing values
- D. Outliers only

Answer: B

20. VIF formula is:

- A. $1 + R^2$
- B. $1/(1 - R^2)$
- C. $R^2/(1 - R^2)$
- D. $1/R^2$

Answer: B

21. A VIF close to 1 generally indicates:

- A. Little multicollinearity
- B. Severe multicollinearity
- C. Missing data
- D. High accuracy

Answer: A

22. Feature engineering can:

- A. Inject domain knowledge
- B. Always guarantee accuracy
- C. Eliminate the need for validation
- D. Eliminate overfitting automatically

Answer: A

23. Which feature might be useful in credit risk?

- A. Loan/income ratio
- B. Customer ID
- C. Random number
- D. Row number

Answer: A

24. Aggregation features are especially useful for:

- A. Transaction/customer data
- B. Removing labels
- C. Image compression
- D. Operating systems

Answer: A

25. Which is an aggregation?

- A. Customer age
- B. Total customer spending
- C. Customer ID
- D. City

Answer: B

26. Tree models can provide:

- A. Feature importance
- B. Only linear coefficients
- C. SQL queries
- D. Missing values

Answer: A

27. Which feature should generally NOT be used directly for prediction?

- A. Income
- B. Age

- C. Customer ID
- D. Credit score

Answer: C

28. Feature selection can help:

- A. Reduce dimensionality
- B. Increase irrelevant features
- C. Guarantee no bias
- D. Guarantee 100% accuracy

Answer: A

29. When should a preprocessing/selection operation generally be fitted?

- A. On the entire dataset before splitting
- B. On training data
- C. Only on test data
- D. On production data only

Answer: B

30. The biggest danger of calculating target encoding before train/test splitting is:

- A. Underfitting
- B. Data leakage
- C. Low memory
- D. High bias

Answer: B

Data Scientist Interview Questions

Q1. What is feature engineering?

Answer:

Feature engineering is the process of transforming raw data into meaningful features that improve a machine learning model's ability to learn patterns.

Example:

$$DTI = \frac{LoanAmount}{Income}$$

Instead of giving the model only income and loan amount, DTI directly represents the borrower's debt burden.

Q2. Feature engineering vs feature selection?

Answer:

Feature engineering creates or transforms variables, while feature selection chooses the most useful variables from an existing feature set.

Q3. Why is feature engineering important?

Answer:

Because raw variables don't always represent the relationships relevant to the prediction task. Well-designed features can improve model performance, interpretability and generalization.

Q4. What is data leakage?

Answer:

Data leakage occurs when information that would not be available at prediction time, or information from the validation/test set, influences model training.

Example:

```
Python
encoder.fit_transform(entire_dataset)
```

before train/test splitting can leak information.

Q5. Why shouldn't you perform feature selection before splitting the data?

Answer:

Because the selection process could use information from the test set. This produces an overly optimistic evaluation.

Correct:

```
Split
↓
Fit selector on train
↓
Transform train
↓
Transform test
```

Q6. When would you use L1 regularization?

Answer:

When I want a sparse model and potentially automatic feature selection. L1 can shrink some coefficients to zero.

Q7. What is multicollinearity?

Answer:

Multicollinearity occurs when independent variables are strongly correlated with each other, which can make coefficient estimates unstable, particularly in linear models.

Q8. How do you detect multicollinearity?

Answer:

Common approaches include:

- Correlation matrix
 - VIF
 - Domain knowledge
-

Q9. Correlation vs mutual information?

Answer:

Correlation primarily measures linear association, while mutual information can capture broader statistical dependency, including nonlinear relationships.

Q10. What is target encoding and what is its biggest risk?

Answer:

Target encoding replaces a categorical value with a statistic derived from the target, such as its mean target rate.

Its biggest risk is **target leakage**.

It should be performed using only appropriate training data/folds.

Q11. Why might you create a log transformation?

Answer:

To reduce right skewness, compress extreme values and sometimes make the relationship between a feature and target more suitable for a model.

Q12. Why can polynomial features cause overfitting?

Answer:

Increasing polynomial degree creates many additional features and allows the model to fit increasingly complex patterns, including noise.

Top 10 Interview Points to Memorize

1. Feature engineering = create/transform features.
2. Feature selection = choose useful features.
3. Log transformation helps skewed distributions.
4. One-hot encoding converts categories into binary columns.
5. Target encoding has leakage risk.
6. Mutual information can detect nonlinear dependency.
7. RFE is a wrapper method.
8. L1 can perform embedded feature selection.
9. VIF detects multicollinearity.
10. Fit preprocessing on training data, then transform test data.

Day 4 Practical Challenge

Without looking at the solution, build a feature-engineering pipeline for a **loan-default prediction dataset** containing:

```
age
income
loan_amount
employment_years
dependents
city
previous_defaults
application_date
default
```

Create at least **10 engineered features**, including:

1. Loan-to-income ratio
2. Income per dependent
3. Log income
4. Age group
5. Employment group
6. Previous-default flag
7. Application month
8. Application weekday

9. City encoding
10. Loan burden category

Then:

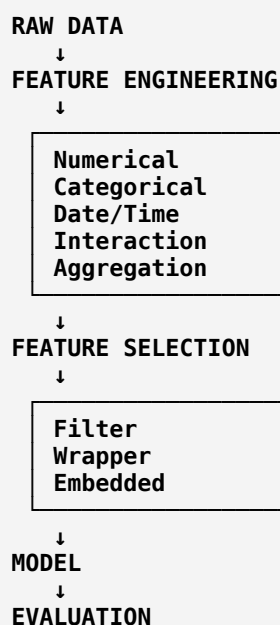
```
↓
Train/Test Split
↓
Feature Selection
↓
Logistic Regression
↓
Evaluate
```

Your goal is not simply to make the model run.

Your goal is to be able to explain **why each feature exists and why it should help the business problem.**

Day 4 Summary

Today's progression:



★ The most important concept today

A Data Scientist doesn't simply feed every available column into a model.

A strong Data Scientist asks:

"What does this variable represent, when would it be available, what information does it contain, and does it help the model solve the actual business problem without leakage?"

That mindset is much more important than memorizing individual feature-engineering techniques.

Day 4 complete: Feature Engineering & Feature Selection.

ChatGPT can make mistakes. Check important info.